



南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

人工智能内容生成 (AIGC) 技术与应用



概率模型的建立与利用

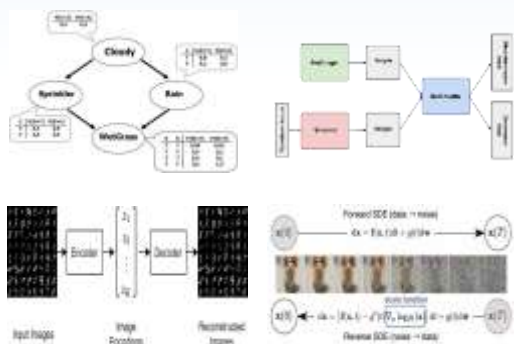
表示

学习

推断

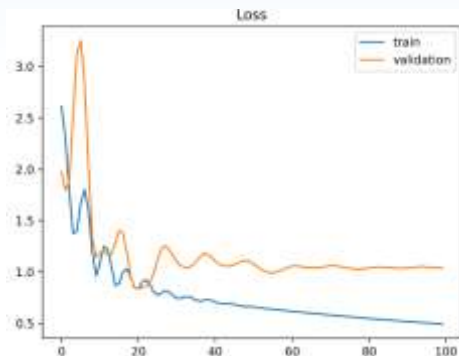
采样

01



选定分布族来定义
和描述数据的分布

02



调整分布族参数以
逼近真实数据分布

03



$P(x)=?$

给定新的观测，求解
出现概率

04



采样向量

从已学到的分布中
随机生成新数据

高斯分布模型的建立与利用

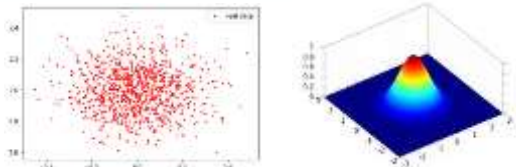
表示

学习

推断

采样

01



用高斯分布来定义
和描述数据的分布

02

$$\ell(\mu, \sigma^2) = \sum_{i=1}^n \ell(x_i | \mu, \sigma^2) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log \sigma^2 - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

$$\hat{\mu}, \hat{\sigma}^2 = \arg \max_{\mu, \sigma^2} \ell(\mu, \sigma^2)$$

求解参数（均值方
差）逼近真实数据

03

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})^2$$

给定新的观测，
求解高斯分布概率

04

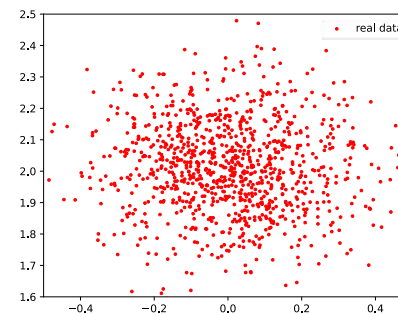
$$Z \sim \mathcal{N}(0, 1).$$

$$X = \mu + \sigma Z$$

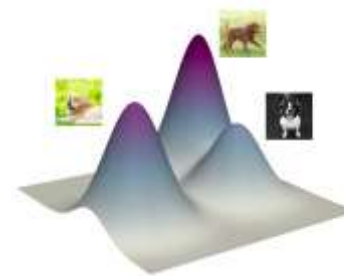
从高斯分布中采样
新样本

真实数据——高维空间的复杂分布

- 如何**表示**复杂分布？——通常不符合常见分布函数
- 如何**学习**复杂分布的参数？——没有闭式解
- 如何**推断**概率？——概率密度函数未知
- 如何**采样**？——计算机难以进行复杂分布的采样



二维点——高斯分布

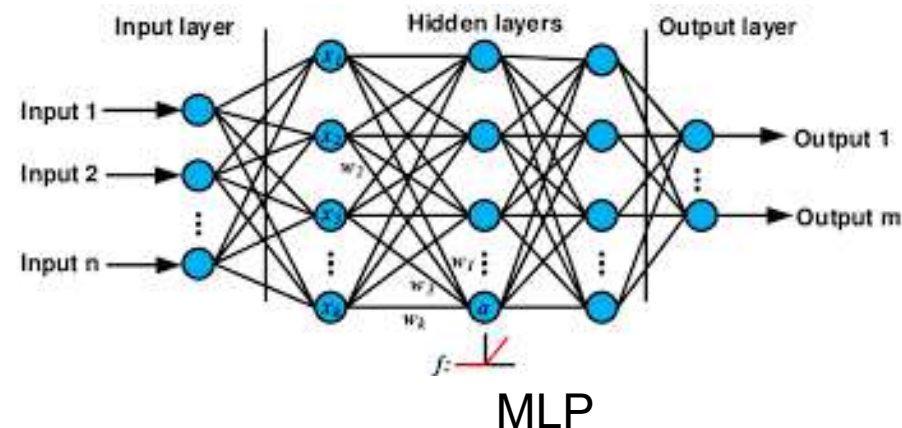


图像——256*256维未知形式的复杂分布

回顾

深度学习：多层神经网络的复合，构建复杂函数映射

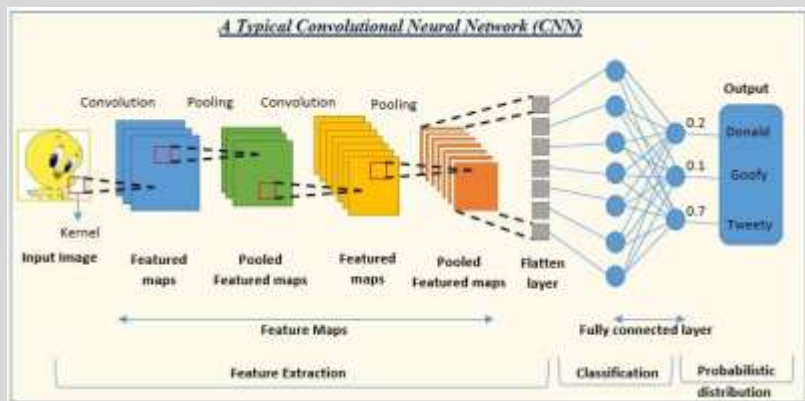
- **深度**：多层复合，更高表达能力。
- **学习**：找到适当的参数 θ



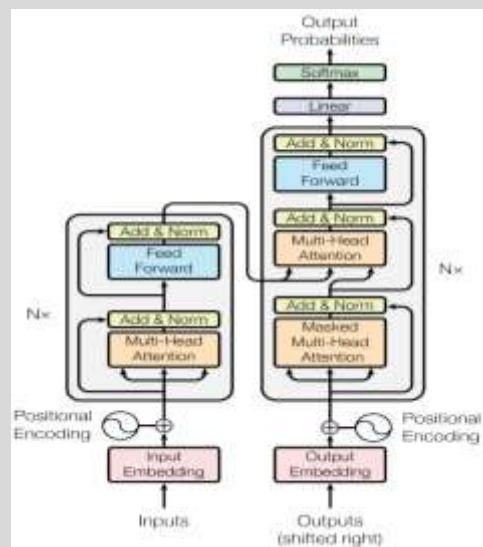
深度学习两大法宝：能力强、好求解

- **Universal Approximation原理**：只要深度足够深，可逼近任意函数
- **梯度回传算法**：给定优化目标，可以高效计算出任意复杂网络的最优参数

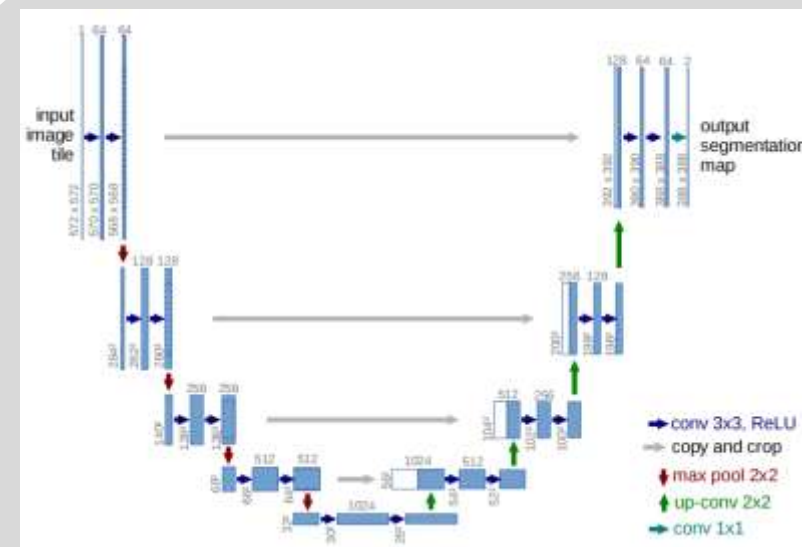
不止 MLP——更多网络结构



CNN



Transformer



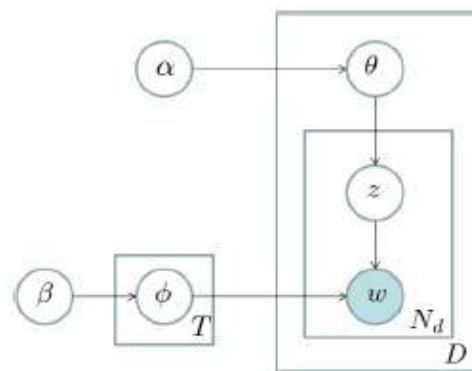
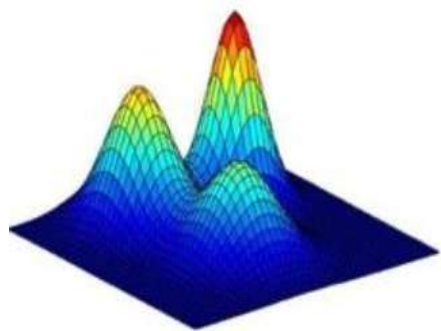
U-Net

目录

- 第一节 概率模型
- 第二节 深度学习
- 第三节 深度生成模型

使用GMM/概率图建模高维空间的复杂分布

核心思想： 简单分布组合/嵌套拟合复杂分布形式

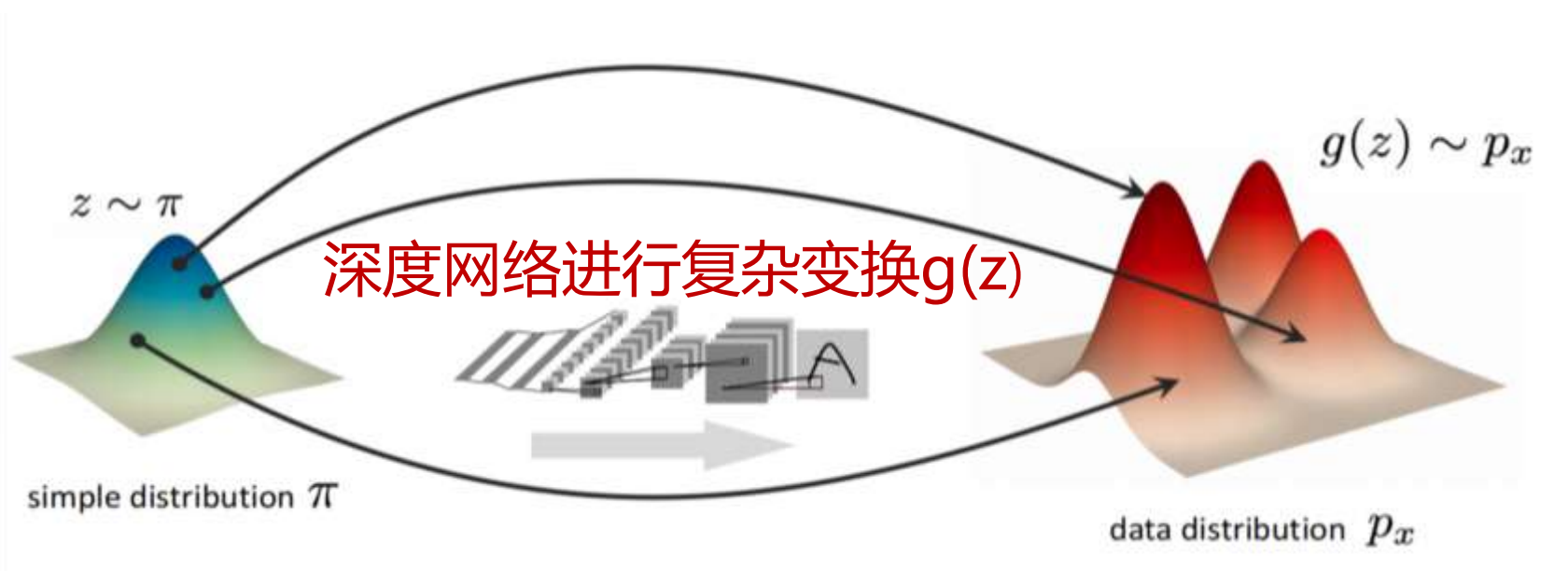


强调对于真实数据分布形式的近似

- 拟合能力受限
- 需要人为设计嵌套方式
- 分布参数难以求解

使用深度学习建模高维空间的复杂分布

核心思想：简单分布采样 + 复杂映射 = 复杂分布采样



简单分布采样的数据

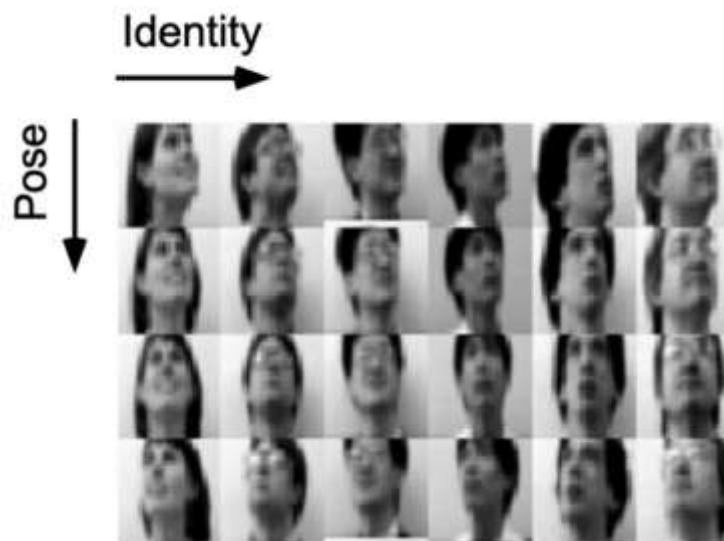
$g(z)$ 服从复杂的目标分布

使用深度学习建模高维空间的复杂分布

核心思想：简单分布采样 + 复杂映射 = 复杂分布采样

➤ 举例（简单分布映射到复杂分布）：

- x 为真实图片，具有复杂分布
- 每个图像 x 对应隐变量 z （姿势、光照、尺度）
- z 符合高斯分布
- z 通过某种“**世界模型**”渲染为 x

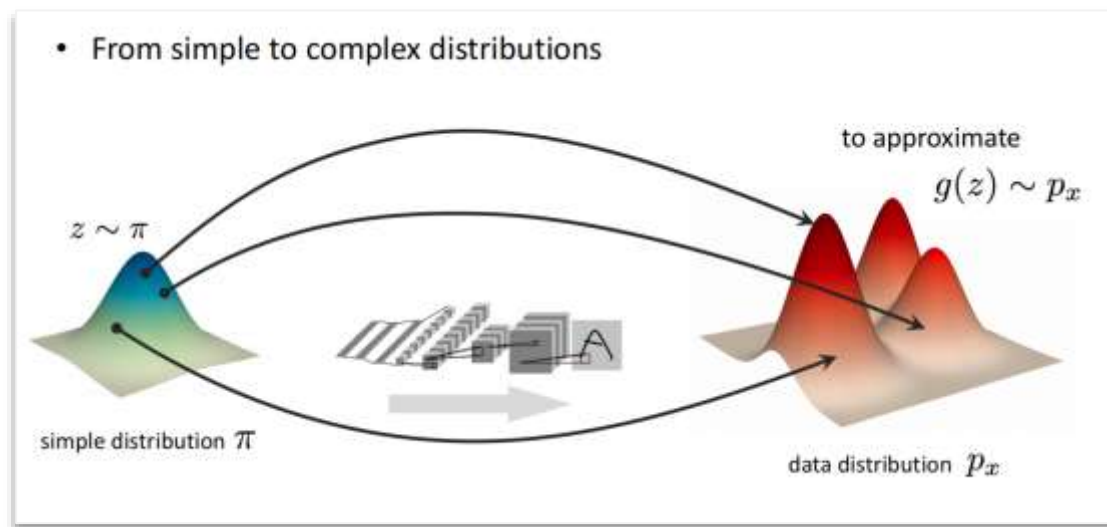


深度学习建模“世界模型”

真实数据——高维空间的复杂分布

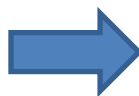
- 如何**表示**复杂分布？——通常不符合常见分布函数
- 如何**学习**复杂分布的参数？——没有闭式解
- 如何推断概率？——概率密度函数未知
- 如何**采样**？——计算机难以进行复杂分布的采样

使用深度学习建模高维空间的复杂分布



➤ 深度学习的优势

- Universal Approximation
- 梯度回传求解
- 确定性算法，无需采样



➤ 深度生成模型优势

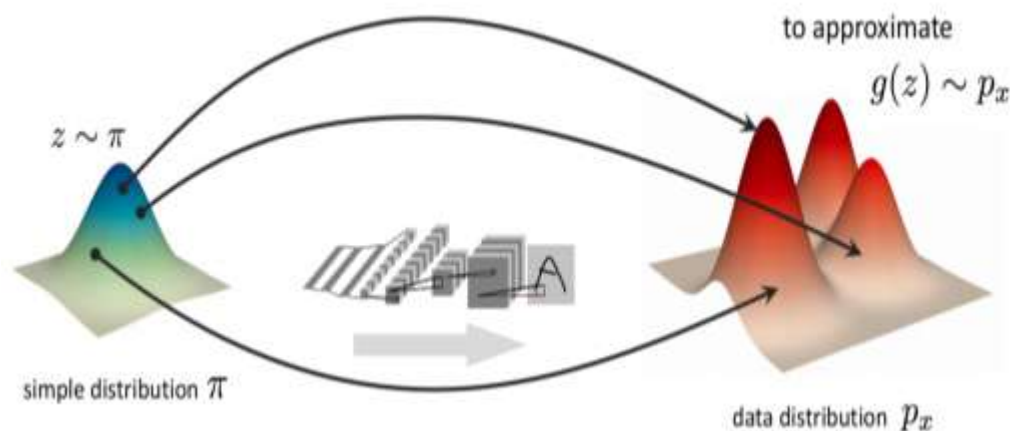
- 表示：表达任意复杂分布的潜力
- 学习：高效求解分布参数
- 采样：只对简单分布进行采样

如何基于深度学习构建生成式模型

简单分布采样 + 复杂映射 = 复杂分布

还有什么问题？

- ① 如何约束 z 和 x 之间的一一对应关系
- ② 如何保证 z 服从简单的分布

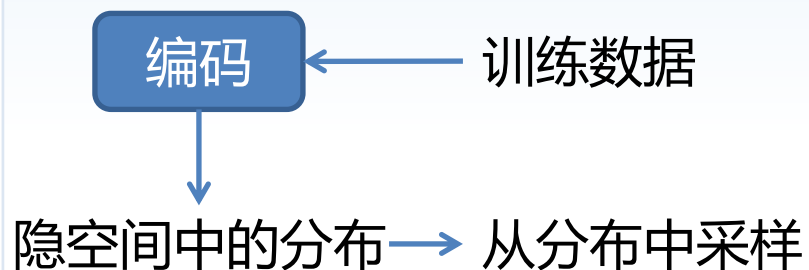


如何解决这两个问题？首先解决哪个问题？
导致了不同生成式方法的不同

如何用深度学习建模高维空间的复杂分布

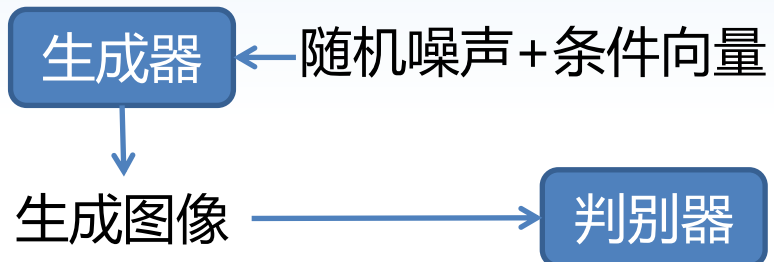
1

VAE



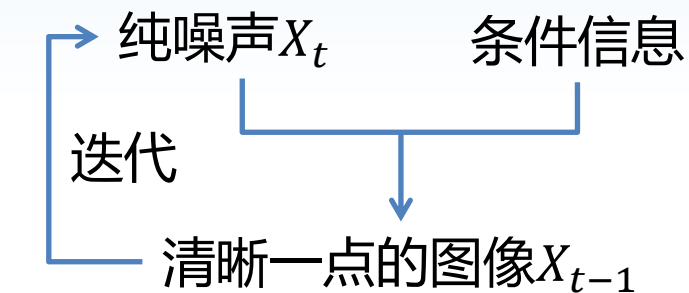
2

GAN



3

扩散模型





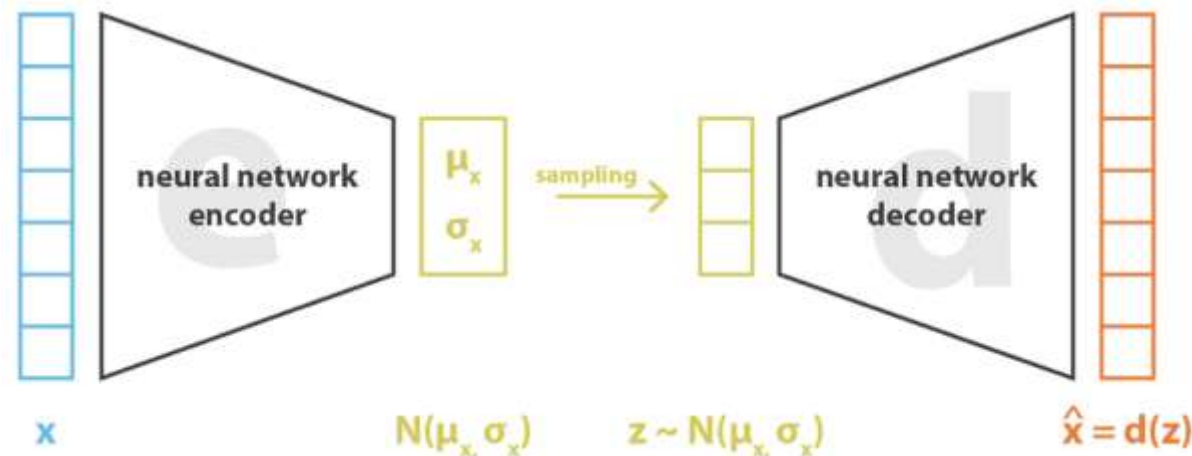
第三章

变分自编码器 (VAE)

变分自编码器

本章目标： 探索变分自编码器如何构建 “简单分布采样+复杂映射”

- **自编码器：** 构建隐变量 z 和样本 x 之间的一一映射。
- **变分：** 确保 z 为简单的高斯分布。





第三章 变分自编码器 (VAE)

目录

- 第一节 自编码器基础(AE)
- 第二节 变分自编码器(VAE)
- 第三节 VQ-VAE

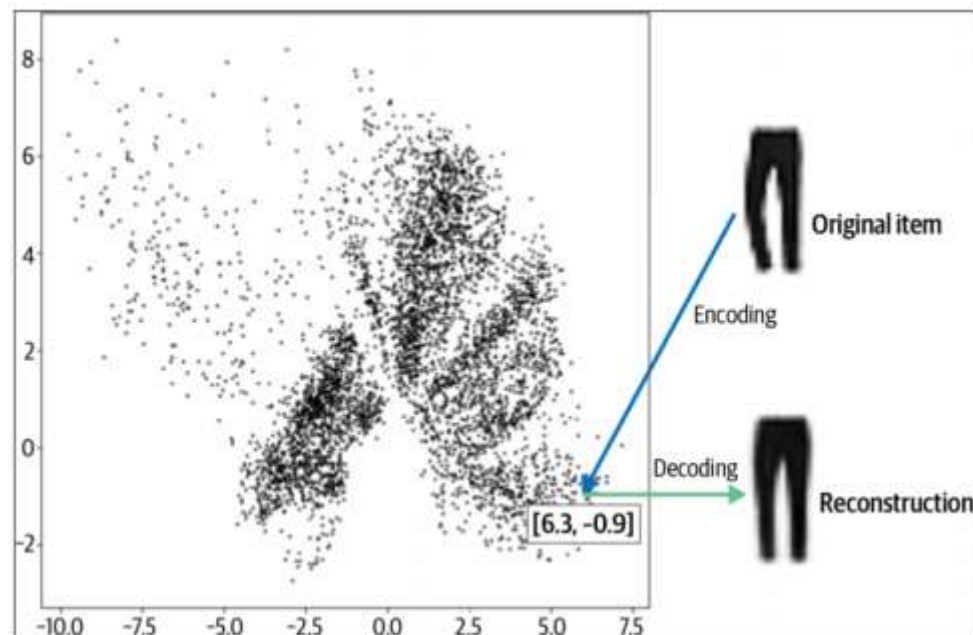
自编码器是什么

自编码器由一对编码器和解码器构成，编码器负责构建高维输入 x 到低维变量 z 的映射，解码器负责从低维变量 z 重构出样本 x

- **直观理解：**将混乱的衣堆整理到一个无限高、无限宽的衣柜里，当想要某件衣服时，只需用位置就可以找到对应衣服。



衣柜——隐空间



衣柜坐标-衣服构建了一一映射关系

自编码器

自编码器是什么

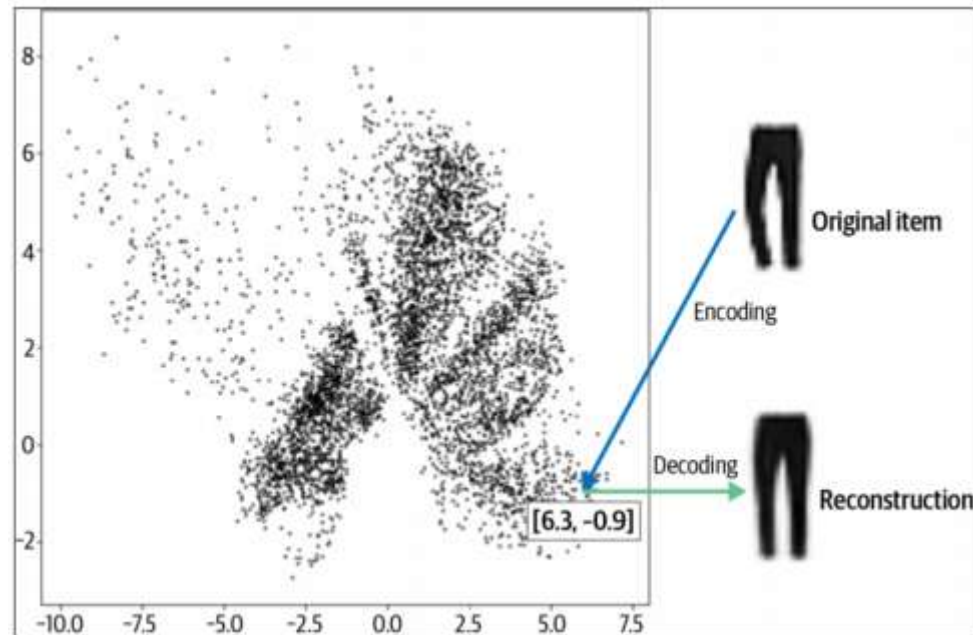
自编码器由一对编码器和解码器构成，编码器负责构建高维输入 x 到低维变量 z 的映射，解码器负责从低维变量 z 重构出样本 x

➤ 直观理解

- **编码器**：整理衣服分类存放在衣橱里的过程
- **解码器**：基于衣橱坐标，取出衣服的过程



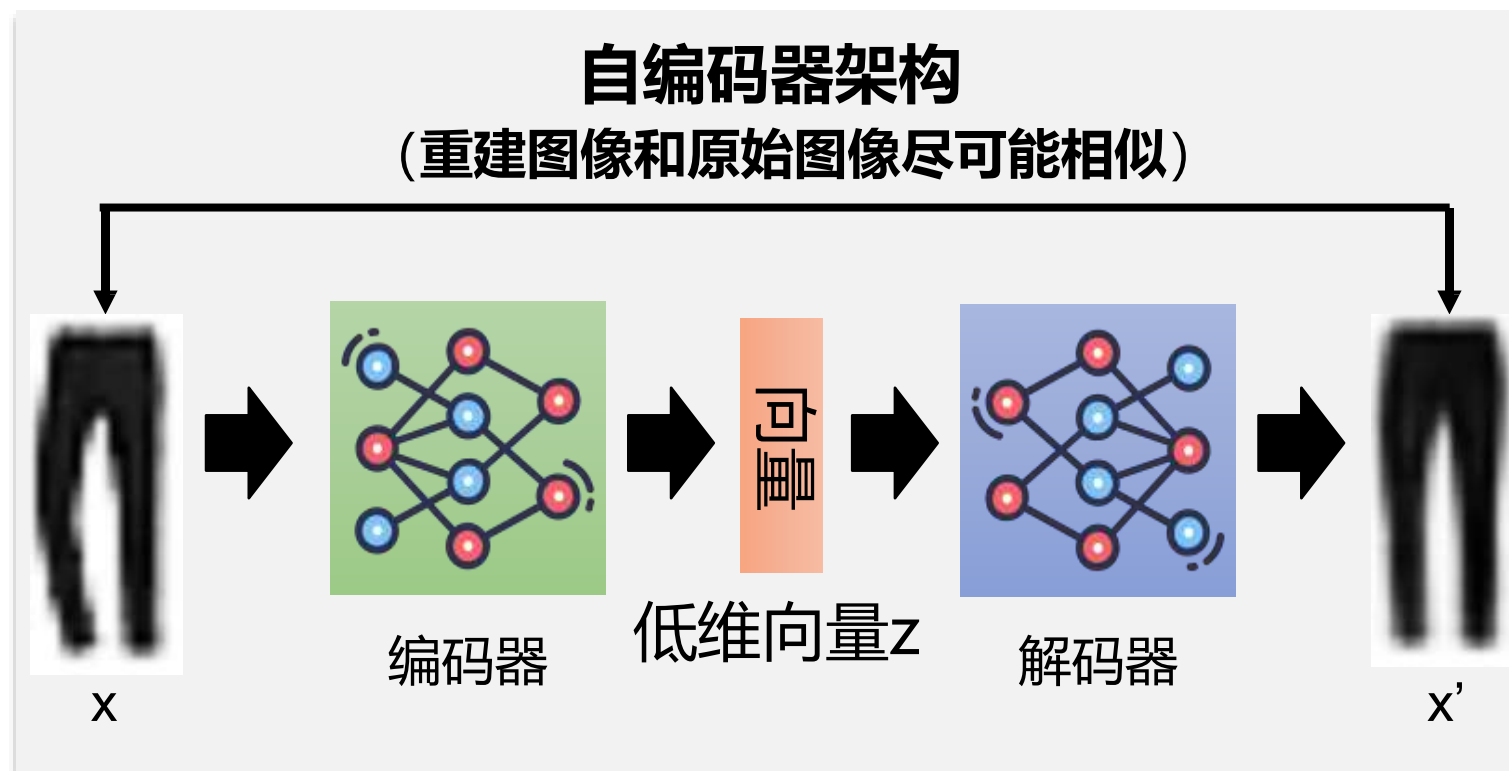
衣橱——隐空间



衣橱坐标-衣服构建了一一映射关系

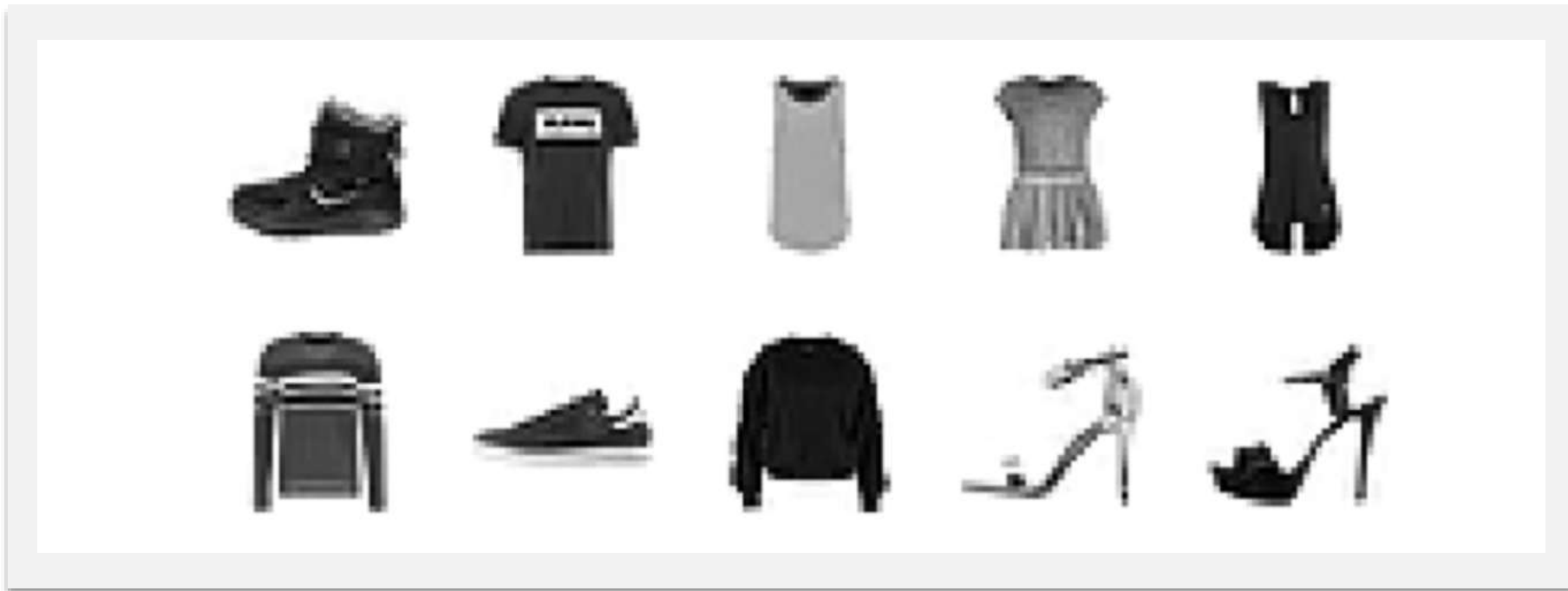
自编码器如何构建？

- 构建自编码器需要明确三方面设置：
- **编码器网络设置：** 压缩表征
- **解码器网络设置：** 解码重构
- **联合优化：** 模型学习



案例：在Fashion-MNIST数据集上构建自编码器

- 10类衣服
- 总共7000张图像
- 28*28分辨率
- 灰度图像数值



```
from tensorflow.keras import datasets  
(x_train,y_train), (x_test,y_test) = datasets.fashion_mnist.load_data()
```


案例：在Fashion-MNIST数据集上构建自编码器

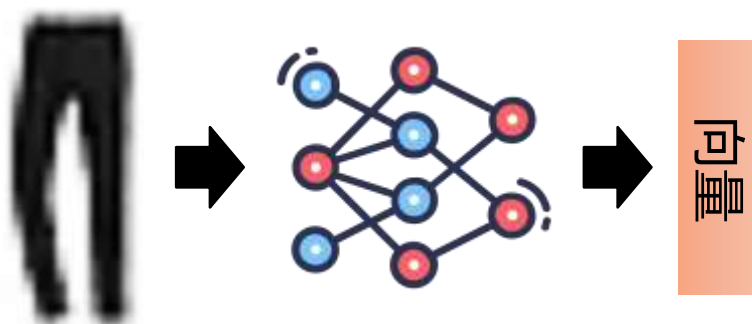
➤ 图像预处理

- 归一化
- 边缘填充
- 增加通道维度

```
def preprocess(imgs):  
    imgs = imgs.astype("float32") / 255.0  
    imgs = np.pad(imgs, ((0, 0), (2, 2), (2, 2)), constant_values=0.0)  
    imgs = np.expand_dims(imgs, -1)  
    return imgs  
  
x_train = preprocess(x_train)  
x_test = preprocess(x_test)
```

构建编码器

编码器: 构建一个神经网络输入是32*32的图像, 输出是2 维向量



Layer (type)	Output shape	Param #
InputLayer	(None, 32, 32, 1)	0
Conv2D	(None, 16, 16, 32)	320
Conv2D	(None, 8, 8, 64)	18,496
Conv2D	(None, 4, 4, 128)	73,856
Flatten	(None, 2048)	0
Dense	(None, 2)	4,098

Total params	96,770
Trainable params	96,770
Non-trainable params	0

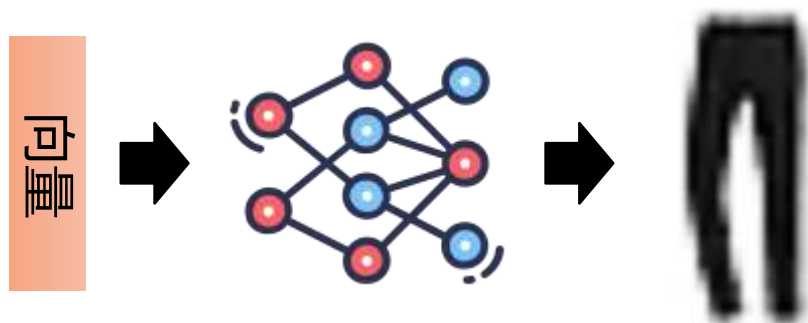
```
encoder_input = layers.Input(
    shape=(32, 32, 1), name = "encoder_input"
) ❶
x = layers.Conv2D(32, (3, 3), strides = 2, activation = 'relu', padding="same")(
    encoder_input
) ❷
x = layers.Conv2D(64, (3, 3), strides = 2, activation = 'relu', padding="same")(x)
x = layers.Conv2D(128, (3, 3), strides = 2, activation = 'relu', padding="same")(x)
shape_before_flattening = K.int_shape(x)[1:]

x = layers.Flatten()(x) ❸
encoder_output = layers.Dense(2, name="encoder_output")(x) ❹

encoder = models.Model(encoder_input, encoder_output) ❺
```

构建解码器

解码器: 构建一个神经网络输入是2维向量, 输出是32*32 的图像



Layer (type)	Output shape	Param #
InputLayer	(None, 2)	0
Dense	(None, 2048)	6,144
Reshape	(None, 4, 4, 128)	0
Conv2DTranspose	(None, 8, 8, 128)	147,584
Conv2DTranspose	(None, 16, 16, 64)	73,792
Conv2DTranspose	(None, 32, 32, 32)	18,464
Conv2D	(None, 32, 32, 1)	289

Total params 246,273
Trainable params 246,273
Non-trainable params 0

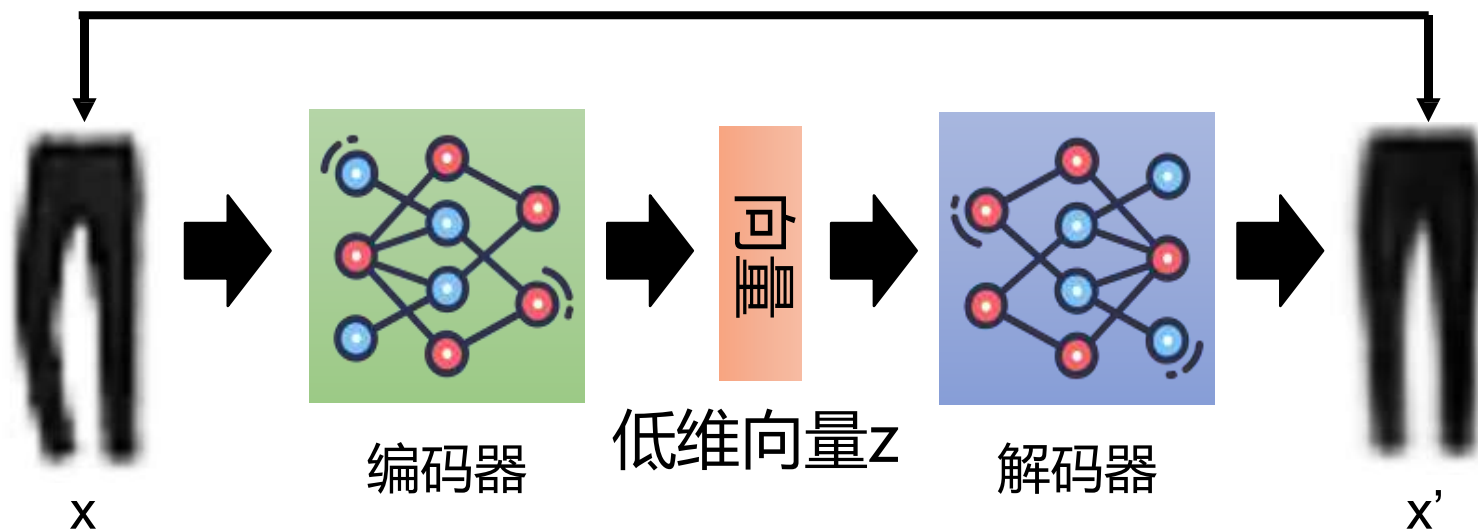
```
decoder_input = layers.Input(shape=(2,), name="decoder_input") ①
x = layers.Dense(np.prod(shape_before_flattening))(decoder_input) ②
x = layers.Reshape(shape_before_flattening)(x) ③
x = layers.Conv2DTranspose(
    128, (3, 3), strides=2, activation = 'relu', padding="same"
)(x) ④
x = layers.Conv2DTranspose(
    64, (3, 3), strides=2, activation = 'relu', padding="same"
)(x)
x = layers.Conv2DTranspose(
    32, (3, 3), strides=2, activation = 'relu', padding="same"
)(x)
decoder_output = layers.Conv2D(
    1,
    (3, 3),
    strides = 1,
    activation="sigmoid",
    padding="same",
    name="decoder_output"
)(x)

decoder = models.Model(decoder_input, decoder_output) ⑤
```

编码器-解码器联合训练

训练目标函数: $\text{loss} = \sigma_x |x - x'|$

(重建图像和原始图像尽可能相似)



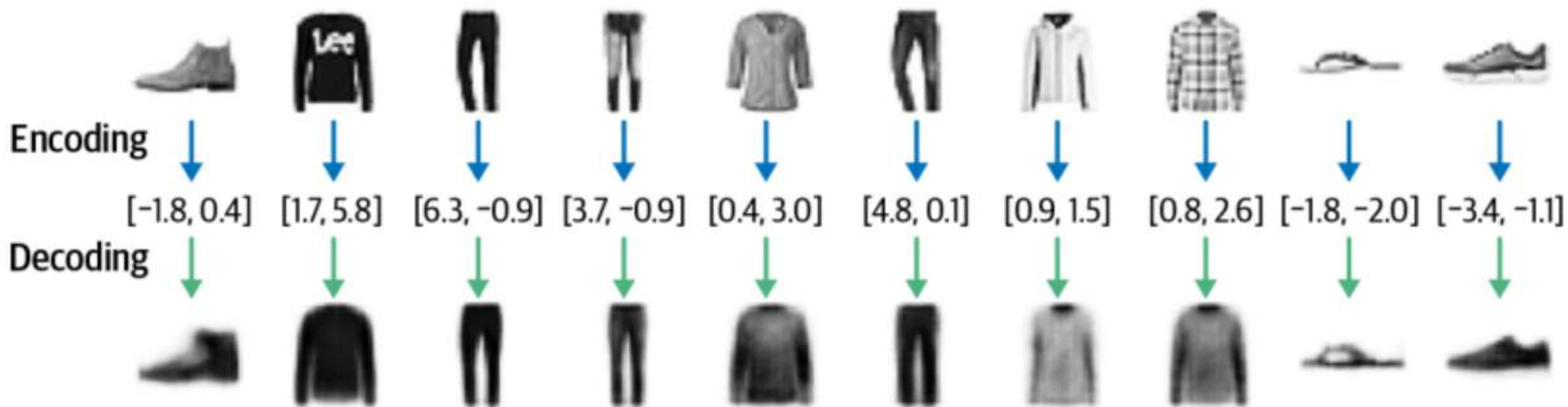
```
autoencoder = Model(encoder_input, decoder(encoder_output)) ❶  
autoencoder.compile(optimizer="adam", loss="binary_crossentropy")
```

书中使用交叉熵损失也是一种选择

编码器-解码器联合训练

```
autoencoder.fit(  
    x_train,  
    x_train,  
    epochs=5,  
    batch_size=100,  
    shuffle=True,  
    validation_data=(x_test, x_test),  
)
```

自编码器隐变量推理

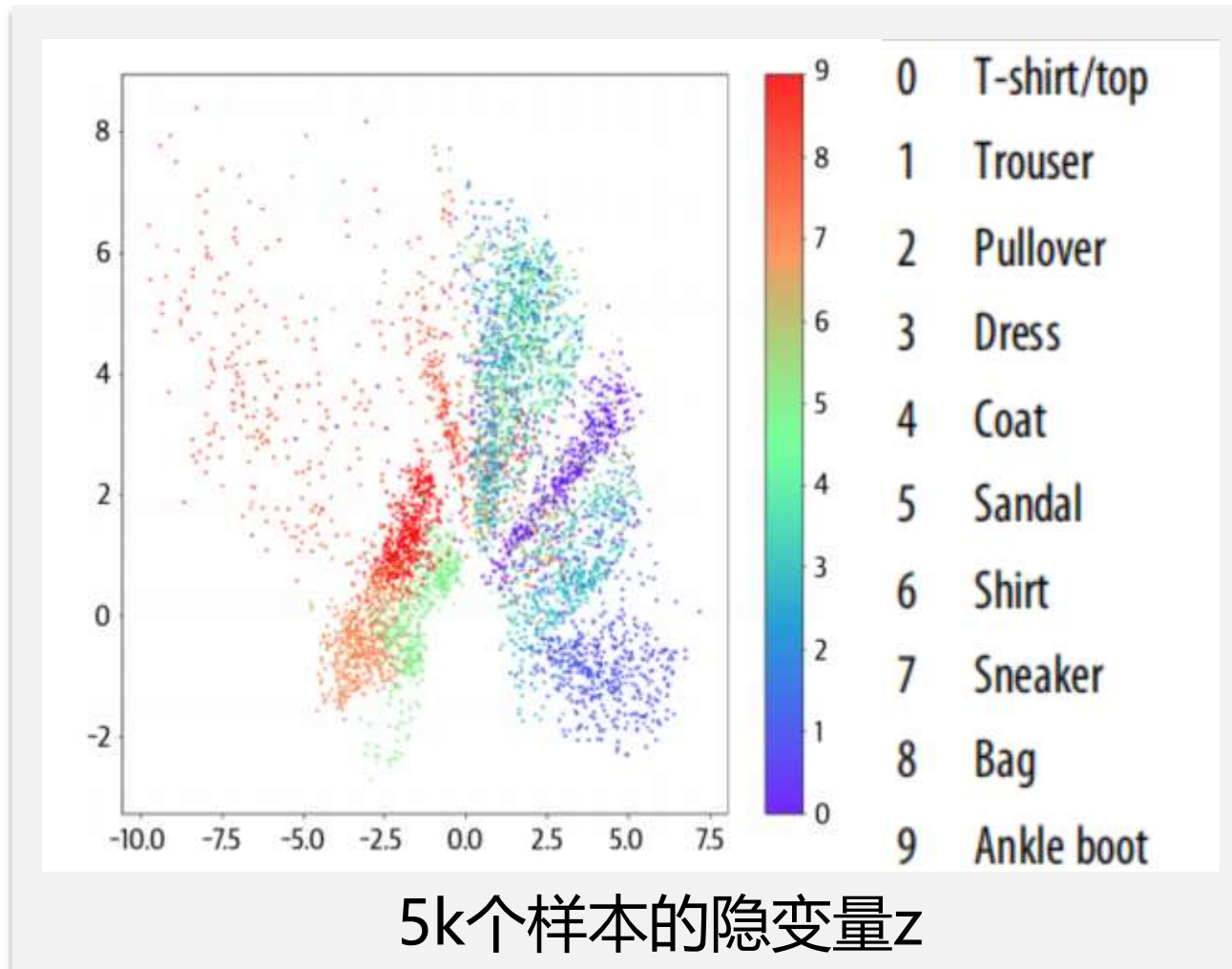


不同衣服可以对应不同隐变量 z :构建了 $x-z-x$ 的映射

思考：自编码器中 z 是一个简单分布吗？

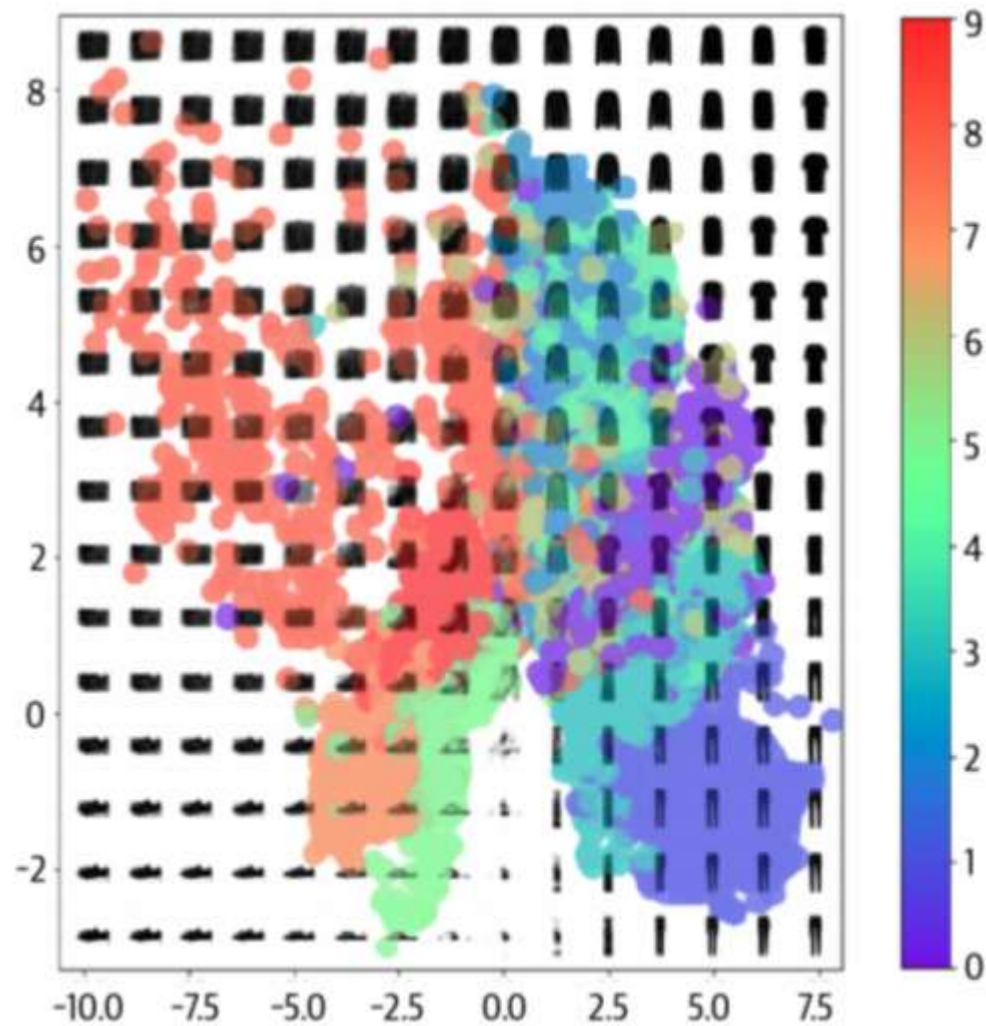
自编码器隐变量可视化

- 观察：分布依然不规则，不是简单分布
 - 问题：自编码器能否实现生成呢？
- 图像中空白区域采集一个点输入到解码器，会生成一个新的样本吗？

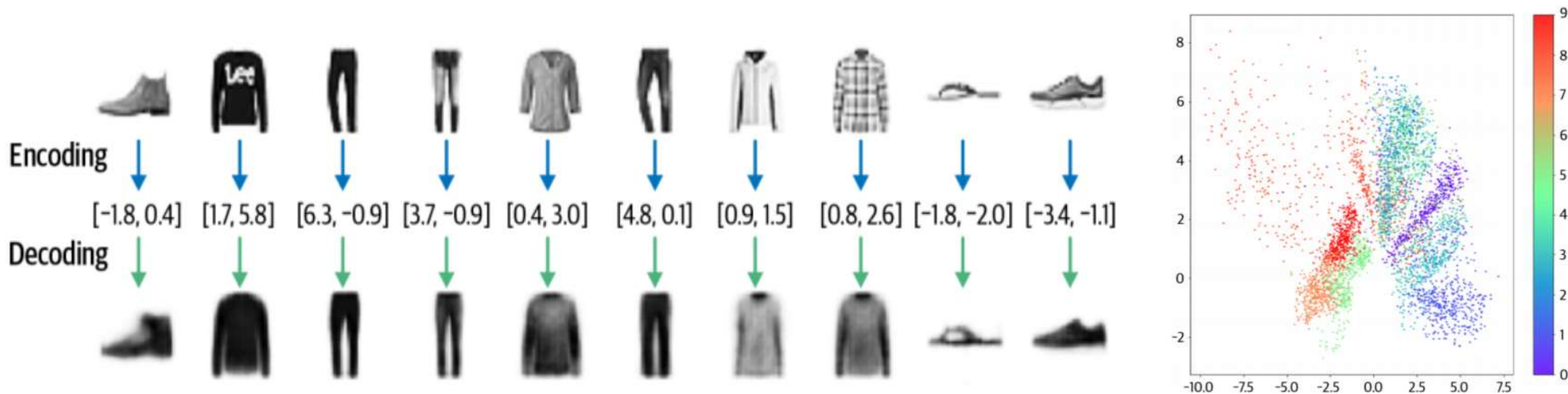


自编码器用于生成新样本

- **不同类别面积差异:**部分衣物密集聚集, 部分则广泛分布。
- **不同类别间隙无意义:**不同类别间存在明显空隙与稀疏点, 对应样本无意义。



自编码器用于生成新样本



结论： 自编码通过压缩表征可以实现一个很好的样本 x 到隐变量 z 的映射，但是并不能实现一个简单良好的隐变量分布，也无法生成新样本

如何构建一个良好的隐变量 z

- 一个良好隐变量要满足的要求：
 - **分布简单**: 比如标准正态分布
 - **相邻坐标对应的样本相似**: 局部语义光滑性

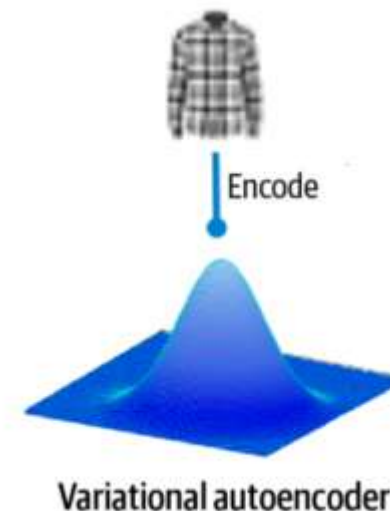
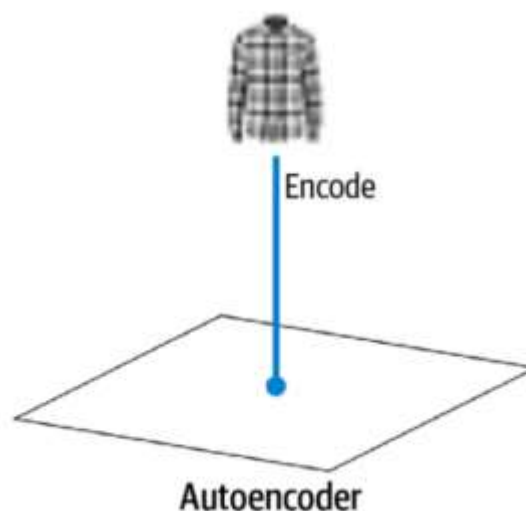
变分自编码器如何通过 “变分” 的方法解决满足上述两个要求?

变分自编码器

自编码器的拓展--变分自编码器 (VAE)

自编码器：建立衣柜坐标与衣物之间的映射关系。

变分自编码器：不仅建立映射，还为每件衣物定义一个相似的临近区域，用**高斯分布**表示。

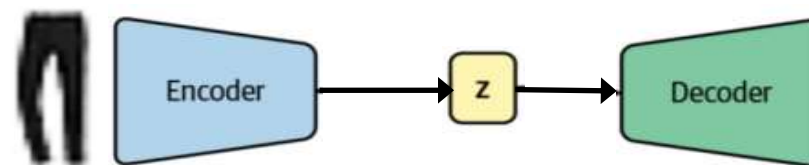


变分自编码器

变分自编码器与自编码器的不同

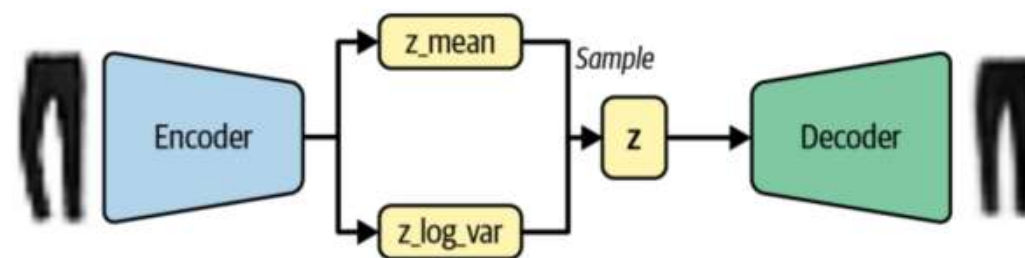
➤ 传统自编码器

- 编码阶段：输入数据 x 被编码为隐变量 z 。
- 解码阶段：将隐变量 z 解码为重构数据 x'



➤ 变分自编码器

- 编码阶段：输入数据 x 被编码为概率分布 $q(z | x)$ 。
- 采样阶段：从分布 $q(z|x)$ 中采样隐变量 z 。
- 解码阶段：将隐变量 z 解码为重构数据 x'

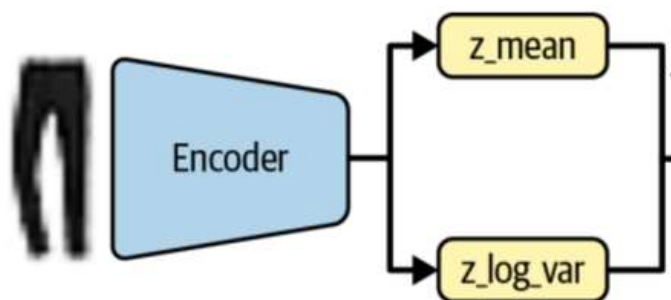


$$z \sim N(z_{mean}, Z_{var})$$

变分自编码器

构建编码器

编码器: 构建一个神经网络输入是32*32的图像，输出两个是2维向量



数学表示:

$$q(z|x) = \mathcal{N}(\mu, \sigma^2)$$

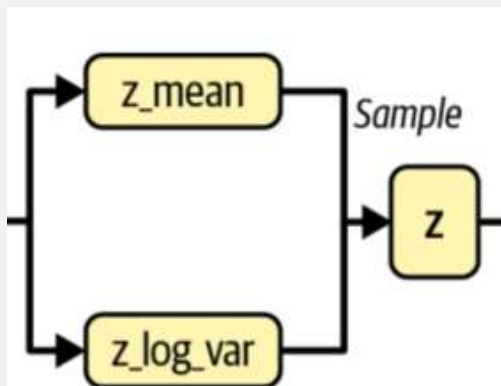
其中, $\mu = f_{\mu}(x)$, $\sigma = f_{\sigma}(x)$, f_{μ} 和 f_{σ} 是神经网络

```
encoder_input = layers.Input(
    shape=(32, 32, 1), name="encoder_input"
)
x = layers.Conv2D(32, (3, 3), strides=2, activation="relu", padding="same")(
    encoder_input
)
x = layers.Conv2D(64, (3, 3), strides=2, activation="relu", padding="same")(x)
x = layers.Conv2D(128, (3, 3), strides=2, activation="relu", padding="same")(x)
shape_before_flattening = K.int_shape(x)[1:]

x = layers.Flatten()(x)
z_mean = layers.Dense(2, name="z_mean")(x) ①
z_log_var = layers.Dense(2, name="z_log_var")(x)
z = Sampling()([z_mean, z_log_var]) ②
```

注解: 网络输出使用logvar便于数值计算

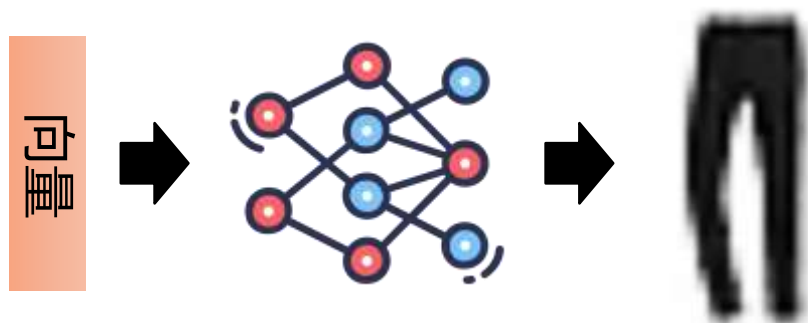
隐变量采样阶段：从 $N(z_{mean}, z_{var})$ 中采样 z



```
class Sampling(layers.Layer): ❶
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = K.random_normal(shape=(batch, dim))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon ❷
```


构建解码器

解码器: 构建一个神经网络输入是2维向量, 输出是32*32 的图像



Layer (type)	Output shape	Param #
InputLayer	(None, 2)	0
Dense	(None, 2048)	6,144
Reshape	(None, 4, 4, 128)	0
Conv2DTranspose	(None, 8, 8, 128)	147,584
Conv2DTranspose	(None, 16, 16, 64)	73,792
Conv2DTranspose	(None, 32, 32, 32)	18,464
Conv2D	(None, 32, 32, 1)	289

Total params 246,273
Trainable params 246,273
Non-trainable params 0

```
decoder_input = layers.Input(shape=(2,), name="decoder_input") ①
x = layers.Dense(np.prod(shape_before_flattening))(decoder_input) ②
x = layers.Reshape(shape_before_flattening)(x) ③
x = layers.Conv2DTranspose(
    128, (3, 3), strides=2, activation = 'relu', padding="same"
)(x) ④
x = layers.Conv2DTranspose(
    64, (3, 3), strides=2, activation = 'relu', padding="same"
)(x)
x = layers.Conv2DTranspose(
    32, (3, 3), strides=2, activation = 'relu', padding="same"
)(x)
decoder_output = layers.Conv2D(
    1,
    (3, 3),
    strides = 1,
    activation="sigmoid",
    padding="same",
    name="decoder_output"
)(x)

decoder = models.Model(decoder_input, decoder_output) ⑤
```

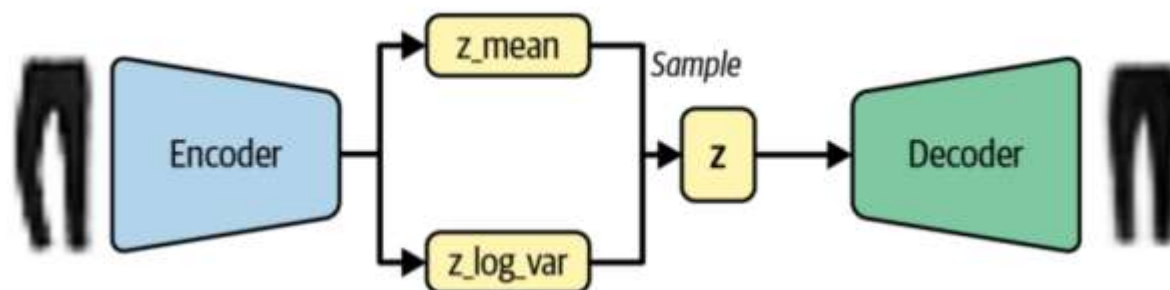

变分自编码器

如何学习参数?

- 目标1: x 和 x' 尽可能相似, 即 $|x - x'|$ 需要尽可能小。
- 目标2: 隐变量 z 对应分布要尽量简单, 即约束隐变量 z 的分布和一个简单分布相似。

损失函数: $\mathcal{L} = \Sigma_x \{|x - x'| + KL(N(\mu_x, \sigma_x^2) | N(0, 1))\}$

其中, KL 散度用于度量两个分布之间的距离。





变分自编码器

➤ 什么是KL散度?

KL散度是一种衡量两个概率分布之间差异的指标，其公式为：

$$D_{KL}(P||Q) = \int p(x) \log \frac{P(x)}{Q(x)} dx$$

其中 $p(x)$ 和 $Q(x)$ 是两个分布。

➤ 高斯分布KL散度的计算

对于两个高斯分布：

$$P(x) = \mathcal{N}(\mu_1, \sigma_1^2), Q(x) = \mathcal{N}(\mu_2, \sigma_2^2)$$

其中KL散度公式为： $D_{KL}(P||Q) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$

- KL散度的特点
- 非对称性： $D_{KL}(P||Q) \neq D_{KL}(Q||P)$
- 非负性： $D_{KL}(P||Q) \geq 0$, 当且仅当 $P(x) = Q(x)$ 时取等号

$$KL(N(\mu, \sigma^2) || N(0, I))$$



$$D_{KL}[N(\mu, \sigma || N(0, 1))] = -\frac{1}{2} \sum (1 + \log(\sigma^2) - \mu^2 - \sigma^2)$$



编码器-解码器联合训练

➤ 构建VAE Class

```
class VAE(models.Model):  
    def __init__(self, encoder, decoder, **kwargs):  
        super(VAE, self).__init__(**kwargs)  
        self.encoder = encoder  
        self.decoder = decoder  
        self.total_loss_tracker = metrics.Mean(name="total_loss")  
        self.reconstruction_loss_tracker = metrics.Mean(  
            name="reconstruction_loss"  
        )  
        self.kl_loss_tracker = metrics.Mean(name="kl_loss")
```

@property

```
def metrics(self):  
    return [  
        self.total_loss_tracker,  
        self.reconstruction_loss_tracker,  
        self.kl_loss_tracker,  
    ]
```

```
def call(self, inputs): ❶  
    z_mean, z_log_var, z = encoder(inputs)  
    reconstruction = decoder(z)  
    return z_mean, z_log_var, reconstruction
```

```
def train_step(self, data): ❷  
    with tf.GradientTape() as tape:  
        z_mean, z_log_var, reconstruction = self(data)  
        reconstruction_loss = tf.reduce_mean(  
            500  
            * losses.binary_crossentropy(  
                data, reconstruction, axis=(1, 2, 3)  
            )  
        ) ❸  
        kl_loss = tf.reduce_mean(  
            tf.reduce_sum(  
                -0.5  
                * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var)),  
                axis = 1,  
            )  
        )  
        total_loss = reconstruction_loss + kl_loss ❹  
  
    grads = tape.gradient(total_loss, self.trainable_weights)  
    self.optimizer.apply_gradients(zip(grads, self.trainable_weights))  
  
    self.total_loss_tracker.update_state(total_loss)  
    self.reconstruction_loss_tracker.update_state(reconstruction_loss)  
    self.kl_loss_tracker.update_state(kl_loss)  
  
    return {m.name: m.result() for m in self.metrics}
```

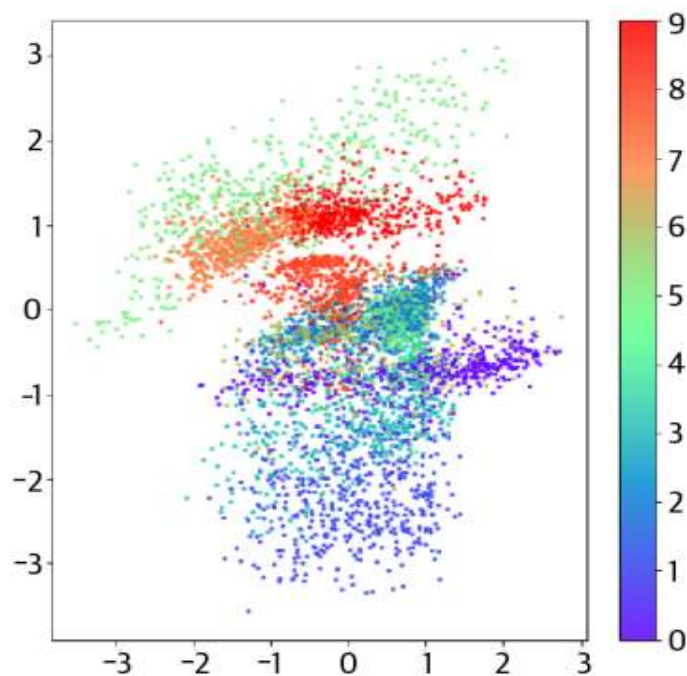


编码器-解码器联合训练

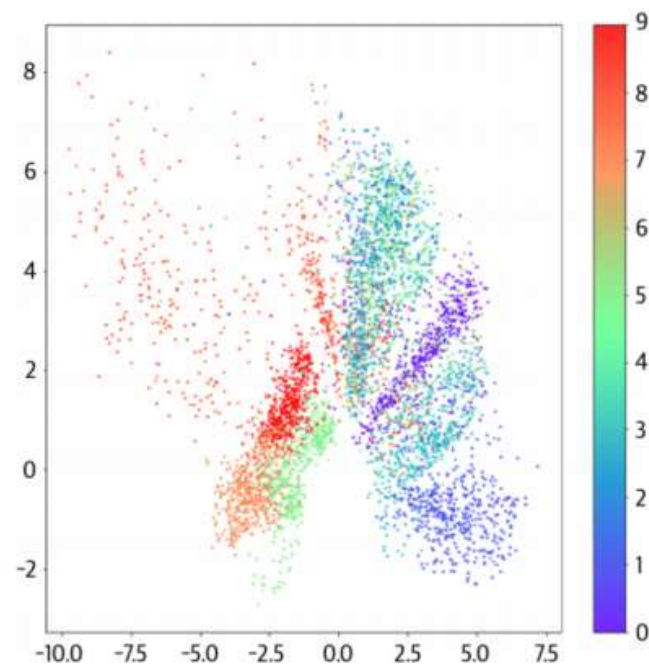
```
vae = VAE(encoder, decoder)
vae.compile(optimizer="adam")
vae.fit(
    train,
    epochs=5,
    batch_size=100
)
```

变分自编码器效果

隐变量空间分布更规整、更像高斯分布



变分自编码器

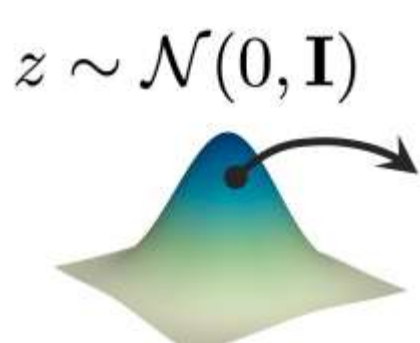


自编码器

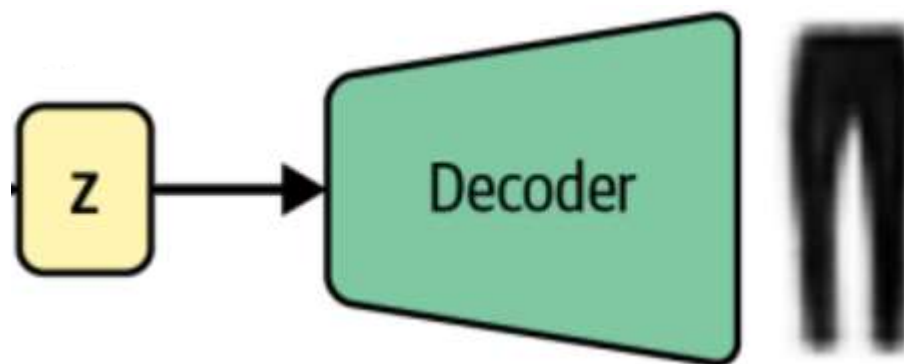
可视化变分自编码器隐层 z 的均值

变分自编码器如何生成新样本呢？

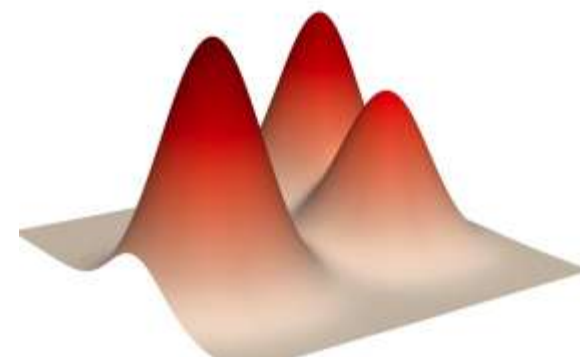
核心思想： 从 $N(0,1)$ 分布中采样隐变量 z 并通过解码器把 z 映射到新样本 x'



(1) 简单分布采样

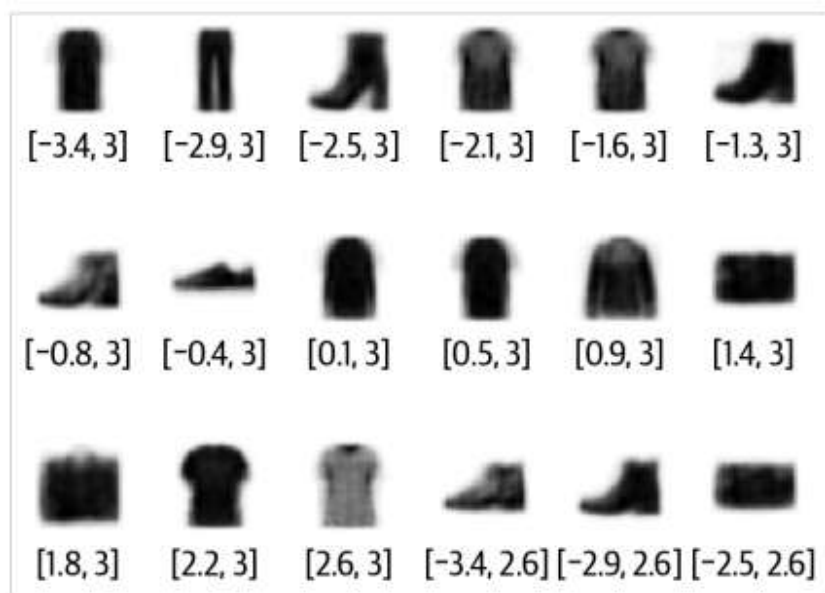


(2) 用解码器映射新样本

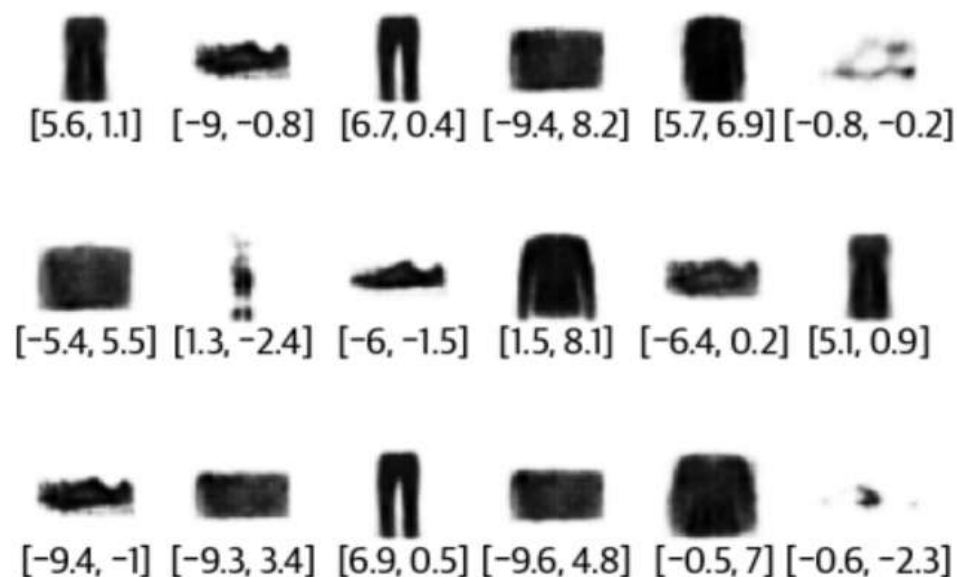


(3) 复杂分布

变分自编码器的效果



变分自编码器

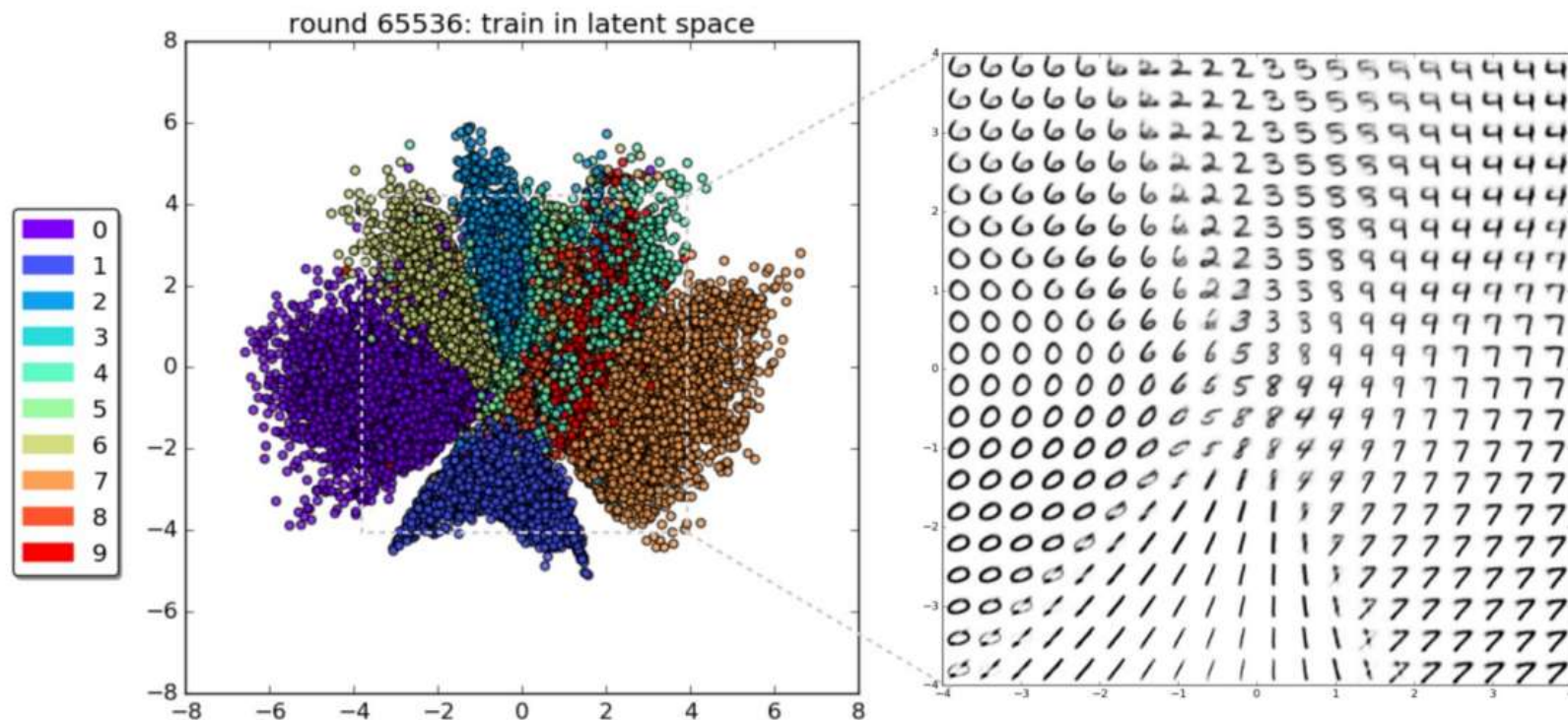


自编码器

新样本生成的质量更高

VAE在MNIST上的效果

- 隐变量空间分布规整
- 隐空间语义连续性强(相邻区域对应图像相似)



变分自编码器

作业

- 基于CelebA数据集构建VAE，并训练完整流程
- 参考代码见课程github
- 要求：
 - 使用pytorch架构重构代码
 - 提交运行完整的jupyter文件



网盘提交:

<https://pan.nuaa.edu.cn/collection/1ccb50ca9401462d7faf5fd975e41f24>

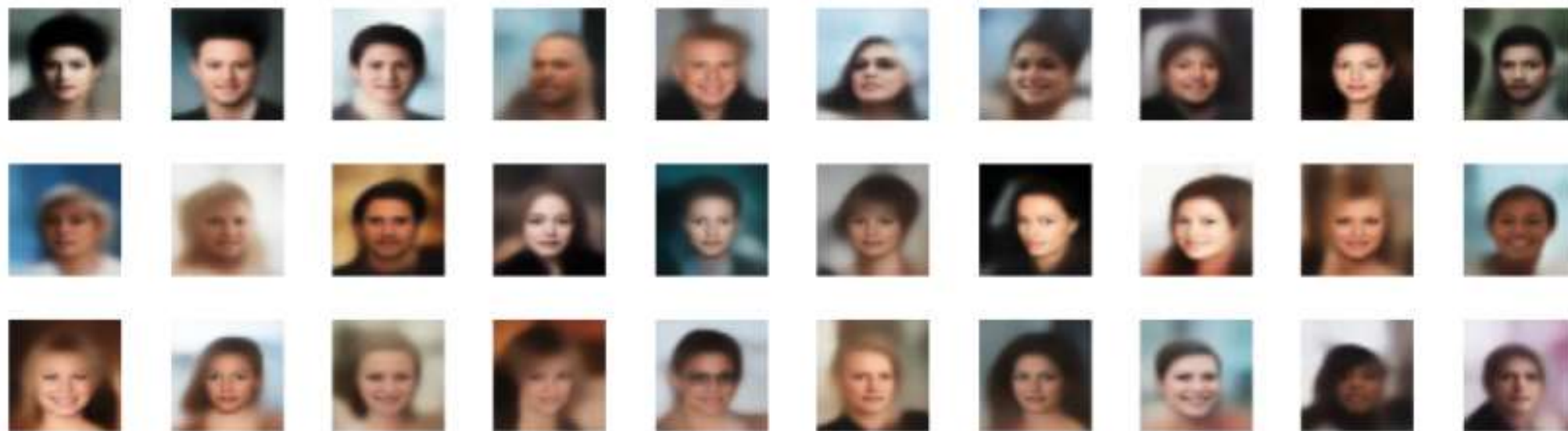
文件命名规则: **homework2-姓名-学号.ipynb**

截止时间: **下周周日**



人脸数据集

预期结果

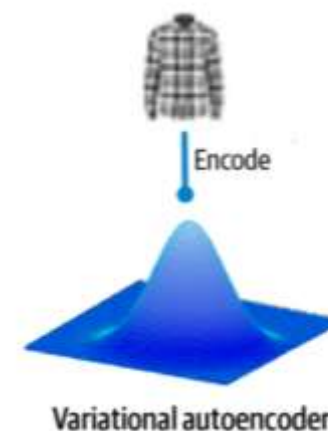


预期生成结果

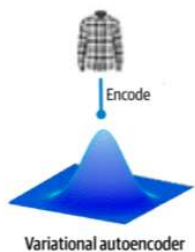
变分自编码器训练目标的另一种理解--最大似然的视角

优化目标：最大似然 $L = \sum_x \log p(x)$

$$\mathcal{L} = \mathbb{E}_x\{|x - x'| + KL(N(\mu_x, \sigma_x^2) | N(0, 1))\}$$



变分自编码器



优化目标：最大似然 $L = \sum_x \log p(x)$

引入 $q(z|x)$, $\int q(z|x) dz = 1$

$$\log p(x) = \int q(z|x) \log p(x) dz$$

贝叶斯定理 $\downarrow p(z, x) = p(x)p(z|x)$

$$\log p(x) = \int q(z|x) \log \frac{p(z, x)}{p(z|x)} dz$$

$\downarrow$ 分解

$$\log p(x) = \int q(z|x) \log \left(\frac{p(z, x)}{q(z|x)} \cdot \frac{q(z|x)}{p(z|x)} \right) dz$$

$$\log p(x) = \int q(z|x) \log \left(\frac{p(z, x)}{q(z|x)} \right) dz + \underbrace{\int q(z|x) \log \left(\frac{q(z|x)}{p(z|x)} \right) dz}_{\substack{KL(q(z|x)|p(z|x)) \\ \text{KL散度} \geq 0}}$$

$$\text{所以 } \log p(x) \geq \underbrace{\int q(z|x) \log \left(\frac{p(z, x)}{q(z|x)} \right) dz}_{L_b, \text{ 变分下界}}$$

变分推断： 用一个简单的分布 $q(z|x)$ 来近似复杂的未知后验分布 $p(z|x)$ 。

变分下界物理意义

$$\log p(x) \geq L_b$$

$$\max_x \sum \log p(x) \quad \Rightarrow \quad \max_x \sum L_b$$

变分下界物理意义

$$L_b = \int q(z|x) \log\left(\frac{p(z, x)}{q(z|x)}\right) dz$$

↓ 贝叶斯定理

$$= \int q(z|x) \log\left(\frac{p(x|z)p(z)}{q(z|x)}\right) dz$$

↓ 拆分

$$L_b = \underbrace{\int q(z|x) \log\left(\frac{p(z)}{q(z|x)}\right) dz}_{-KL(q(z|x)||P(z)), \text{ 正则项}} + \underbrace{\int q(z|x) \log p(x|z) dz}_{\text{重建项}}$$

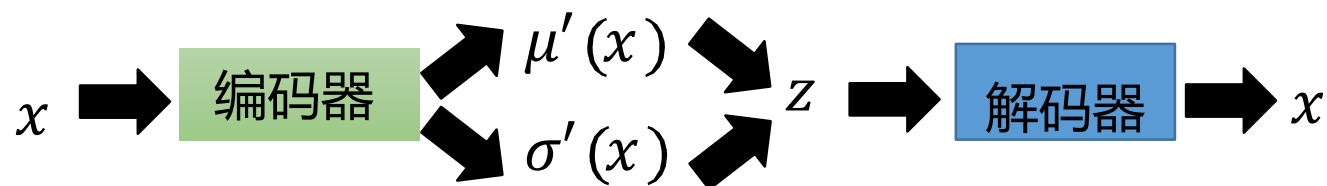
迫使解码器输出接近原始输入

变分下界物理意义

$$\max L_b = -KL(q(z|x)||p(z)) + \int q(z|x) \log p(x|z) dz$$

$$-KL(N(\mu_x, \sigma_x^2)|N(0, 1))$$

最大化 $\int q(z|x) \log p(x|z) dz$



AE的重建损失

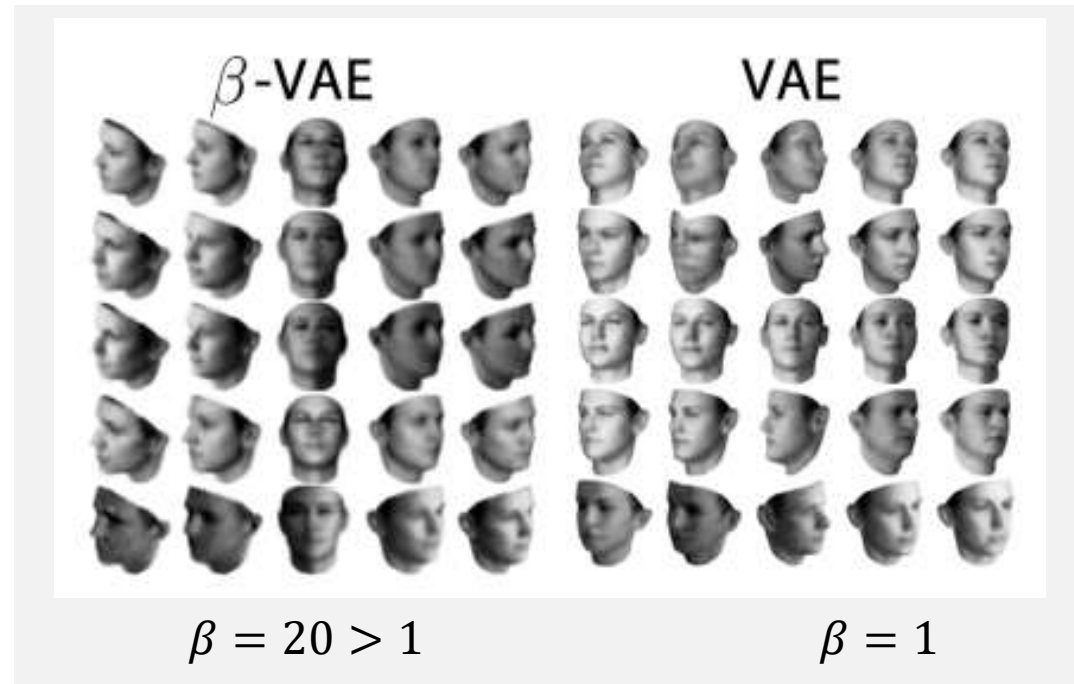
$$-||x - x'||^2$$

$$\min \mathcal{L} = \Sigma_x\{|x - x'| + KL(N(\mu_x, \sigma_x^2)|N(0, 1))\}$$

拓展--KL散度的系数必须是1吗？

$$L = \Sigma_x \{|x - x'| + \beta KL(N(\mu_x, \sigma_x^2) | N(0,1))\}$$

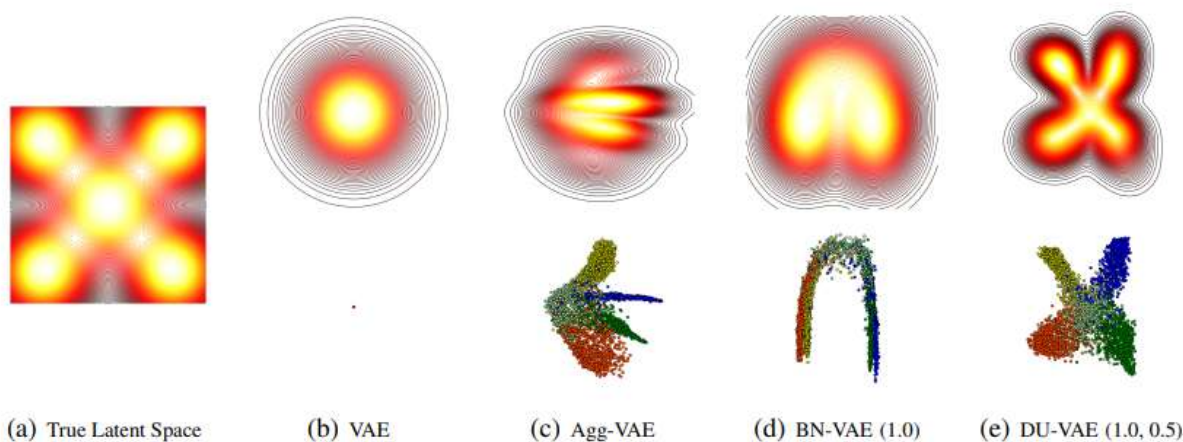
β 越大，隐变量越规整，语义连续性越强，**但重构与生成质量会更差。**



拓展—VAE隐变量分布一定是高斯分布吗？

- 有些情况下，高斯采样与真实数据之间的映射并不好建立
- 模型侧重优化KL损失，导致模型退化
- 可为高斯参数添加约束，让隐变量更容易学

$$\min \mathcal{L} = \Sigma_x \{|x - x'| + KL(N(\mu_x, \sigma_x^2) | N(0, 1))\}$$

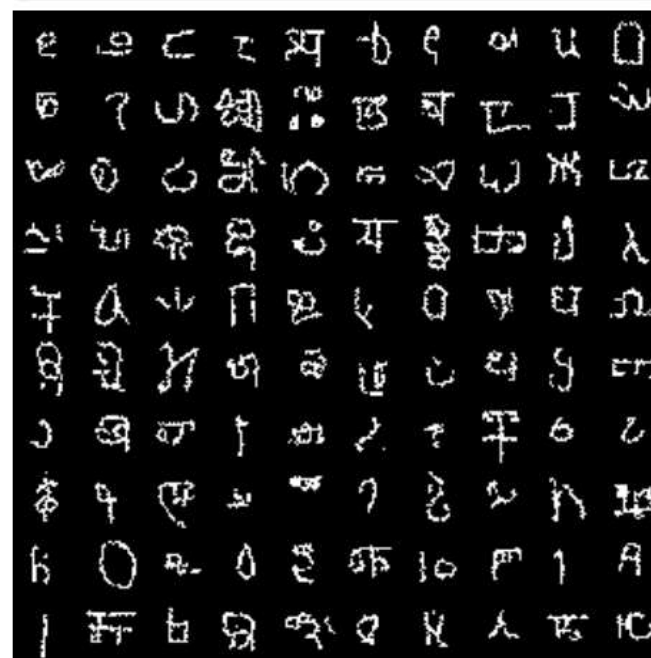


Algorithm 1 Training Procedure of DU-VAE

```

1: Initialize  $\phi, \theta, \gamma_\mu = \gamma$ , and  $\beta_\mu = 0$ 
2: while not convergence do
3:   Sample a mini-batch  $x$ 
4:    $\mu_x, \delta_x^2 = f_\phi(x)$ .
5:    $\hat{\mu}_x = BN_{\gamma_\mu, \beta_\mu}(\mu_x), \hat{\delta}_x^2 = Dropout_p(\delta_x^2)$ .
6:   Sample  $z \sim \mathcal{N}(\hat{\mu}_x, \hat{\delta}_x^2)$  and generate  $x$  from  $f_\theta(z)$ .
7:   Compute gradients  $g_{\phi, \theta} \leftarrow \nabla_{\phi, \theta} \mathcal{L}_{ELBO}(x; \phi, \theta)$ .
8:   Update  $\phi, \theta, \gamma_\mu, \beta_\mu$  according to  $g_{\phi, \theta}$ .
9:    $\gamma_\mu = \frac{\gamma_\mu}{\sqrt{E[\gamma_\mu^2]}} \odot \gamma_\mu$ 
10: end while

```



(a) Samples generated from the prior distribution

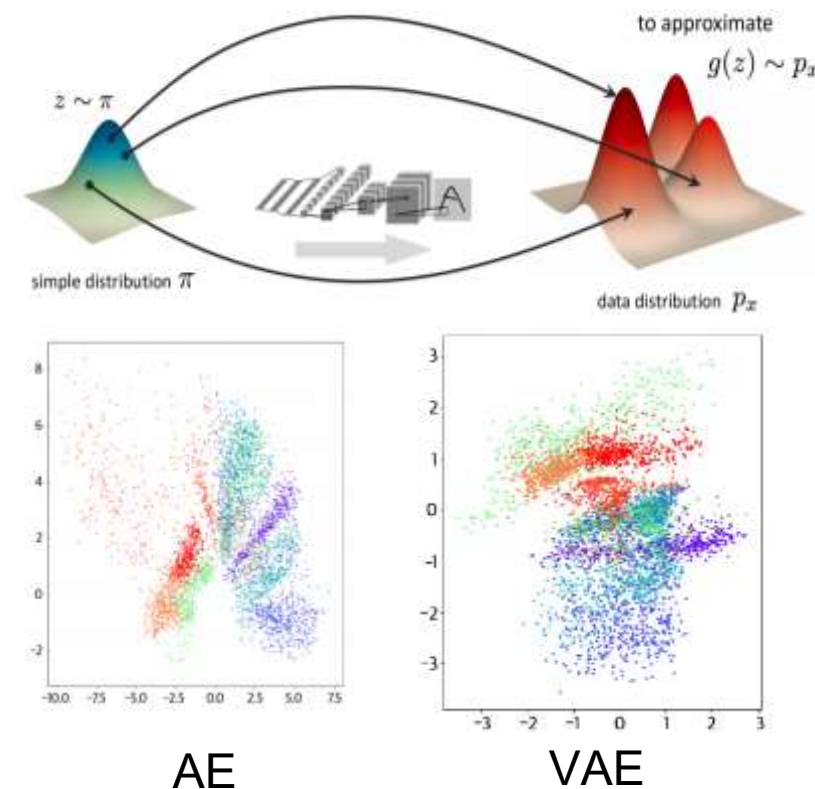


总结

- 深度生成式模型 $p(x)$:
 - **简单分布+复杂映射=复杂分布**
- 自编码器AE由编码器与解码器组合而成，为确定性模型
 - 编码器：从高维输入到低维隐变量 z
 - 解码器：从低维隐变量 z 重构输入样本 x
- **AE实现 z 到 x 的复杂映射，但隐变量 z 的数据分布并不好，无法实现采样生成**

总结

- 变分自编码器VAE是自编码器的变种。
 - 编码器：将高维输入映射到低维隐变量 z 对应的高斯分布 μ, σ
 - 隐变量采样： $z \sim N(\mu, \sigma^2)$
 - 解码器：从低维隐变量 z 重构输入样本 x
- 相比AE， VAE进一步约束隐变量 z 的分布与高斯分布相似



$$\mathcal{L} = \Sigma_x \{ |x - x'| + KL(N(\mu_x, \sigma_x^2) | N(0, 1)) \}$$

复杂映射

简单分布

